UNITÀ
01
LEZIONE
02

I cicli while e for

Iterazioni numeriche in Python

Prof. Francesco Gobbi
Anno scolastico 2026/2027
I.I.S.S. Galileo Galilei – Ostiglia (MN)

La sintassi dei cicli in Python

`while condizione:` La condizione viene valutata prima di ogni esecuzione del blocco. Nel conteggio, inizializziamo il contatore prima del ciclo e lo aggiorniamo nel corpo.

`for nome in intervallo:` Il nome assume, uno alla volta, i valori dell'intervallo. Per lavorare con gli interi, costruiamo l'intervallo con `range()`.

Regole di scrittura

- I **due punti** chiudono l'intestazione del ciclo.
- Il corpo è **indentato**: usiamo quattro spazi per ogni livello.
- Un'istruzione allineata al ciclo si esegue dopo la sua conclusione.

Con `while` gestiamo esplicitamente l'aggiornamento. Con `for` il valore successivo viene assegnato automaticamente.

while: dallo schema al codice

Riprendiamo lo schema iniziale: `while condizione:`

Schema concettuale

```
inizializzazione
while condizione:
    istruzioni del corpo
    aggiornamento
istruzioni successive
```

Esempio in Python

```
numero = 1
while numero <= 5:
    print(numero)
    numero = numero + 1
print("Fine")
```

Inizializzazione: `numero = 1`. **Condizione:** `numero <= 5`.

Corpo: stampa il numero e lo incrementa. La condizione viene ricontrollata prima di ogni iterazione.

Risultato: da 1 a 5, uno per riga, poi `Fine`.

Nota bene – indentazione. Il rientro definisce il corpo del `while`: qui stampa e aggiornamento. Usiamo quattro spazi. `print("Fine")` è fuori dal ciclo.

while: seguire l'esecuzione

Per il ciclo precedente, il controllo avviene **prima** della stampa.

Valore al controllo	`numero <= 5`	**Stampa**	**Dopo l'aggiornamento**
1	`True`	1	2
2	`True`	2	3
3	`True`	3	4
4	`True`	4	5
5	`True`	5	6
6	`False`	Nessuna	Il ciclo termina

Il corpo viene eseguito **5 volte**, la condizione è controllata **6 volte**. Se è falsa all'inizio, il corpo non viene eseguito.

while: conto alla rovescia

```
numero = 5
while numero >= 1:
    print(numero)
    numero = numero - 1
print("Via!")
```

```
OUTPUT  5
        4
        3
        2
        1
        Via!
```

Questa volta il valore **diminuisce** a ogni iterazione.

Si stampa anche 1, perché `1 >= 1` è vero. Poi `numero` diventa 0 e il ciclo termina.

La stampa di `"Via!"` è fuori dal blocco: viene eseguita una sola volta, alla fine.

Accumulare una somma con while

```
numero = 1
somma = 0  # Totale iniziale
while numero <= 4:
    somma = somma + numero
    numero = numero + 1
print("Somma:", somma)
```

```
OUTPUT  Somma: 10
```

`numero` indica il valore corrente: 1, 2, 3, 4.

`somma` è un **accumulatore**: conserva il totale calcolato fino a quel momento.

Ogni passaggio aggiunge il numero corrente al totale. La stampa finale mostra `1 + 2 + 3 + 4 = 10`.

L'accumulatore viene inizializzato **prima** del ciclo. Se assegnassimo `somma = 0` dentro il blocco, perderemmo il totale precedente.

Come cambia l'accumulatore

All'inizio `somma = 0`. Seguiamo il valore del totale a ogni iterazione.

Numero corrente	Somma prima	Calcolo	Somma dopo
1	0	`0 + 1`	1
2	1	`1 + 2`	3
3	3	`3 + 3`	6
4	6	`6 + 4`	10

In `somma = somma + numero`, Python prima calcola il lato destro, poi associa `somma` al risultato. Il valore precedente serve per costruire il nuovo totale.

while: gli errori da riconoscere

Aggiornamento assente Se il valore controllato non cambia, una condizione vera può restare vera per sempre: il ciclo diventa **infinito**.

Direzione sbagliata Con `numero <= 5` e partenza da 1, sottrarre 1 allontana il valore dalla condizione di arresto.

Indentazione errata L'aggiornamento deve stare dentro il ciclo. Se è fuori, viene raggiunto solo dopo la fine del ciclo.

Limite sbagliato Con partenza da 1, `numero < 5` stampa fino a 4; `numero <= 5` include anche 5.

Prima di eseguire un `while`, chiediti: quale valore cambia e quando la condizione diventa falsa? Se il notebook continua a eseguire, usa il comando di interruzione della cella.

range(): quale oggetto viene creato?

`range()` crea un **oggetto di tipo range**: una sequenza immutabile di interi, descritta da inizio, limite escluso e passo.

```
intervallo = range(2, 11, 2)
print(type(intervallo))
print(intervallo)
```

```
OUTPUT  <class 'range'>
        range(2, 11, 2)
```

`intervallo` è il nome che fa riferimento all'oggetto creato.

Inizio: 2 **Limite:** 11
Passo: 2

Valori rappresentati: **2, 4, 6, 8, 10**.

L'oggetto conserva i parametri dell'intervallo e calcola i valori quando servono, senza memorizzarli tutti. Il `for` può percorrerlo e usare un valore alla volta.

range(): inizio, limite e passo

`range()` costruisce un oggetto che rappresenta una sequenza di **interi**. Scegliamo inizio, limite escluso e passo prima di usarlo nel `for`.

Forma	Esempio	Valori forniti
`range(fine)`	`range(4)`	0, 1, 2, 3
`range(inizio, fine)`	`range(1, 5)`	1, 2, 3, 4
`range(inizio, fine, passo)`	`range(2, 9, 2)`	2, 4, 6, 8

Se omettiamo l'inizio, vale `0`. Se omettiamo il passo, vale `1`.

Il limite finale è sempre escluso. Con passo positivo si procede finché il valore è minore del limite; con passo negativo finché è maggiore. Il passo non può essere zero.

range(): creare e osservare un intervallo

```
intervallo = range(2, 11, 2)
print(intervallo)
print(len(intervallo))
```

```
OUTPUT  range(2, 11, 2)
        5
```

L'intervallo rappresenta i valori **2, 4, 6, 8, 10**.

`print(intervallo)` mostra la descrizione dell'oggetto, non elenca i valori.

`len(intervallo)` restituisce il numero di valori rappresentati: in questo caso 5.

`range()` non esegue istruzioni e non è un ciclo. Descrive i valori da percorrere; il `for` userà quei valori per eseguire il proprio corpo.

range(): scegliere i valori da percorrere

Obiettivo	Espressione	Valori
Cinque valori a partire da zero	`range(5)`	0, 1, 2, 3, 4
Da 1 a 5, inclusi	`range(1, 6)`	1, 2, 3, 4, 5
Dispari da 1 a 9	`range(1, 10, 2)`	1, 3, 5, 7, 9
Da 5 a 1, in discesa	`range(5, 0, -1)`	5, 4, 3, 2, 1
Passo incompatibile coi limiti	`range(5, 0)`	Nessun valore

Il limite finale indica dove fermarsi, non un valore da includere.

Prima di scrivere il ciclo, verifica il primo valore, il passo e l'ultimo valore desiderato. `range()` accetta interi; un passo uguale a zero provoca un errore.

for: dallo schema al codice

Riprendiamo lo schema iniziale: `for nome in intervallo:`

Schema concettuale

```
for nome in intervallo:
    istruzioni del corpo
istruzioni successive
```

Esempio in Python

```
for numero in range(1, 6):
    print(numero)
print("Fine")
```

Schema descrittivo: sostituiamo nome, intervallo e istruzioni con quelli del nostro programma.

Nome: `numero`. **Intervallo:** `range(1, 6)`, con valori da 1 a 5.

Corpo: `print(numero)`. Il `for` assegna automaticamente al nome il valore successivo e ripete il corpo.

Risultato: da 1 a 5, uno per riga, poi `Fine`.

Nota bene – indentazione. Il rientro definisce il corpo del `for`: qui `print(numero)`. Usia-

for: passi positivi e negativi

Numeri pari da 2 a 10

```
for numero in range(2, 11, 2):
    print(numero)
```

```
OUTPUT  2
        4
        6
        8
        10
```

Conto alla rovescia

```
for numero in range(5, 0, -1):
    print(numero)
```

```
OUTPUT  5
        4
        3
        2
        1
```

A sinistra il passo `2` fa crescere il valore. A destra il passo `-1` lo fa diminuire. I limiti `11` e `0` non vengono raggiunti dal corpo dei cicli.

for: la tabellina di un numero

```
n = 3
for fattore in range(1, 6):
    prodotto = n * fattore
    print(n, "x", fattore,
          "=", prodotto)
```

```
OUTPUT  3 x 1 = 3
        3 x 2 = 6
        3 x 3 = 9
        3 x 4 = 12
        3 x 5 = 15
```

`n` resta uguale a 3.

`fattore` assume i valori da 1 a 5. A ogni passaggio si calcola un nuovo prodotto e si stampa una riga.

La stampa occupa due righe di codice per leggibilità. Le parentesi aperte permettono di continuare l'istruzione nella riga successiva.

for: sommare da 1 a n

```
n = int(input("Limite: "))
somma = 0
for numero in range(1, n + 1):
    # Aggiungiamo il valore corrente
    somma = somma + numero
print("Somma:", somma)
```

```
SESSIONE Limite: 4
         Somma: 10
```

Inseriamo un intero positivo.

`n + 1` permette di includere `n`. Con `n = 4` il ciclo usa 1, 2, 3 e 4.

La stampa è fuori dal ciclo: mostra solo il totale finale. Dentro il ciclo mostrerebbe anche i totali intermedi.

while e for: lo stesso risultato

Con while

```
numero = 1
while numero <= 3:
    print(numero)
    numero = numero + 1
```

Con for

```
for numero in range(1, 4):
    print(numero)
```

```
OUTPUT  1
        2
        3
```

while rende espliciti valore iniziale, condizione e aggiornamento.

for con range() è comodo quando conosciamo l'intervallo di interi da percorrere. Entrambe le forme possono risolvere questi esercizi numerici.

for: limiti e sequenze vuote

```
print("Primo ciclo")
for numero in range(1, 1):
    print(numero)
print("Secondo ciclo")
for numero in range(3, 0):
    print(numero)
print("Fine")
```

```
OUTPUT  Primo ciclo
        Secondo ciclo
        Fine
```

`range(1, 1)` non fornisce valori: l'inizio coincide con il limite escluso.

`range(3, 0)` usa il passo predefinito +1, che non permette di scendere verso 0.

Per ottenere 3, 2, 1 bisogna scrivere `range(3, 0, -1)`.

Esercizio 1: contare fino a n

Consegna

Leggi un intero `n` compreso tra 1 e 20. Stampa tutti gli interi da 1 a `n`, estremi inclusi, uno per riga.

Richiesta

Usa un ciclo `while`. Dopo il ciclo stampa `Fine conteggio`.

Casi di prova

Con `n = 4`: stampa 1, 2, 3, 4 e poi il messaggio.

Con `n = 1`: stampa 1 e poi il messaggio.

Controlla che il valore finale `n` venga stampato e che il messaggio compaia una sola volta.

Esercizio 2: conto alla rovescia

Consegna

Leggi un intero `n` compreso tra 1 e 10. Stampa i numeri da `n` a 0, includendo entrambi gli estremi.

Richiesta

Usa un ciclo `while`. Dopo lo zero stampa `Partenza!`.

Casi di prova

Con `n = 3`: stampa 3, 2, 1, 0 e poi `Partenza!`.

Con `n = 1`: stampa 1, 0 e poi il messaggio.

Il valore deve diminuire a ogni iterazione. Verifica con attenzione il caso in cui è uguale a zero.

Esercizio 3: numeri pari

Consegna

Stampa tutti i numeri pari da 2 a 20 inclusi, uno per riga.

Richiesta

Usa `for` con `range()` e un passo adatto, senza usare `if`.

Casi di prova

I primi valori devono essere 2, 4, 6.

Gli ultimi devono essere 16, 18, 20.

Le righe numeriche devono essere esattamente 10.

Il limite finale di `range()` è escluso: scegli un limite che permetta di stampare anche 20.

Esercizio 4: tabellina completa

Consegna

Leggi un intero `n` compreso tra 1 e 10. Mostra la sua tabellina, dal fattore 1 al fattore 10.

Richiesta

Usa un ciclo `for`. Ogni riga deve avere la forma `n x fattore = prodotto`.

Casi di prova

Con `n = 4`: la prima riga è `4 x 1 = 4`; l'ultima è `4 x 10 = 40`.

Il programma deve stampare 10 righe.

Il numero scelto resta fisso; cambia il fattore. Calcola e stampa il prodotto dentro il ciclo.

Esercizio 5: somma dei primi interi

Consegna

Leggi un intero `n` compreso tra 1 e 20. Calcola la somma degli interi da 1 a `n`, includendo `n`.

Richiesta

Usa un ciclo `while` e un accumulatore. Stampa solo il risultato finale con l'etichetta `Somma:`.

Casi di prova

Con `n = 5`: `Somma: 15`.

Con `n = 1`: `Somma: 1`.

Con `n = 10`: `Somma: 55`.

Inizializza il totale prima del ciclo e aggiungi il numero corrente prima di passare al successivo.

Esercizio 6: somma dei quadrati

Consegna

Leggi un intero `n` compreso tra 1 e 10. Calcola la somma `1**2 + 2**2 + ... + n**2`.

Richiesta

Usa un ciclo `for` e un accumulatore. Calcola ogni quadrato dentro il ciclo, poi stampa il totale finale.

Casi di prova

Con `n = 3`: `1 + 4 + 9 = 14`.

Con `n = 1`: il totale è 1.

Con `n = 4`: il totale è 30.

`numero ** 2` calcola il quadrato. Somma i singoli quadrati: non calcolare il quadrato della somma.